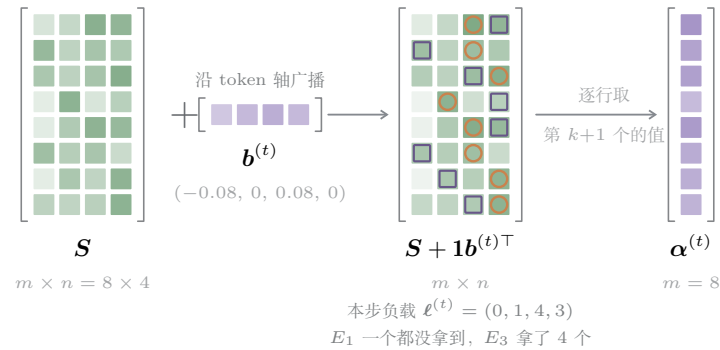


$$\mathcal{T}_i = \operatorname{argtop}_k(\mathbf{s}_i + \mathbf{b}^{(t)}), \quad p_{i,j} = \frac{s_{i,j}}{\sum_{r \in \mathcal{T}_i} s_{i,r}} \quad (\mathbf{b} \text{ 只进 argtop, 不进 } p)$$

$$\widehat{\mathbf{b}}_j^{(t+1)} \leftarrow -\operatorname{quantile}_{1-k/n}(\mathbf{s}_{:,j} - \boldsymbol{\alpha}^{(t)}), \quad \mathbf{b}^{(t+1)} \leftarrow \widehat{\mathbf{b}}^{(t+1)} - \operatorname{mean}(\widehat{\mathbf{b}}^{(t+1)})\mathbf{1}$$

目标负载 $q := mk/n$ 。下图是论文 Fig.5 的同一个小例 $m=8, n=4, k=1, q=2$ ，但换成张量视角；所有数字可用同目录 `qb-numbers.py` 复算。K3 实际是 $n=896, k=16$ ，分位数 ≈ 0.982

(a) 一次 Top-(k+1) 就够——前 k 个是真正路由（**橙圈**），第 $k+1$ 个的值就是门槛 α_i （**紫框**）：一个专家必须超过它才进得了 token i 的 Top- k

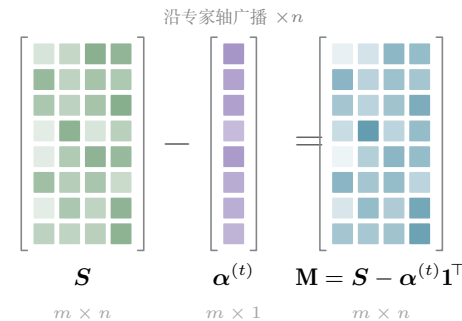


为什么是 Top-(k+1) 而不是 Top- k

Top- k 只告诉你「谁进来了」, QB 还需要知道「差多少才进得来」。把选择改成 Top-($k+1$), 第 $k+1$ 个的分数天然就是这条门槛, **不用再单独算一次 token 侧的分位数**。

门槛取自有偏分数 $\mathbf{s}_i + \mathbf{b}^{(t)}$, 而下一行的 margin 取自原始分数 $\mathbf{s}$ ——旧偏置只通过门槛进入更新, 不会被重复计入。

(b) 把「谁能进」化成逐列的一维问题——每个 token 的门槛沿专家轴广播减掉, 剩下的 margin 只差一个按列的常数就能定胜负

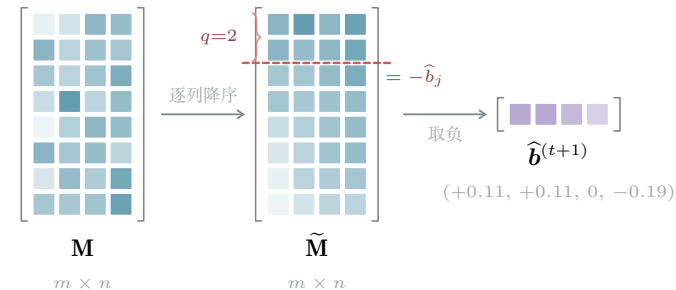


margin 的读法

$M_{ij} = s_{ij} - \alpha_i^{(t)}$: 专家 j 距离挤进 token i 的 Top- k 还差多少。色越深越接近门槛 (按值映射, 不是 $|M|$)。

换上候选偏置 $\widehat{\mathbf{b}}_j$ 之后, 专家 j 拿到 token i 当且仅当 $M_{ij} > -\widehat{\mathbf{b}}_j$ 。于是「专家 j 的负载」= $\sum_i \mathbf{1}[M_{ij} > -\widehat{\mathbf{b}}_j]$, 关于阈值 $-\widehat{\mathbf{b}}_j$ **单调递减**——这就是可以直接求逆的原因。

(c) 逐列降序, 在第 (q+1) 行切一刀——单调 $\Rightarrow$ 可以直接求逆: 要让每列恰好 q 个留在线上, 阈值就只能落在第 (q+1) 大那一格的值上, 而这个秩对四列都一样, 所以是同一条横线

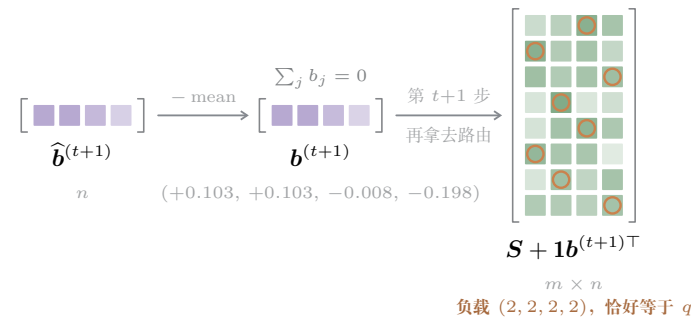


这一刀为什么就是分位数

每列 m 个 margin, 要求恰好 q 个严格大于阈值, 阈值就只能取第 (q+1) 大。四列的秩都是 q+1, 所以是同一条横线——变的只是那一格的值。

又因为 $q/m = k/n$, 「留下 q/m 的比例」正是 $1 - k/n$ 分位数, 于是写成 $\widehat{\mathbf{b}}_j = -\operatorname{quantile}_{1-k/n}(\mathbf{s}_{:,j} - \boldsymbol{\alpha})$ 。K3 里 $1 - 16/896 \approx 0.982$, 是个相当靠右的分位数。

(d) 去掉公共平移, 下一步再用——减去均值不改变任何 Top- k (所有专家同时平移), 只是把 $\mathbf{b}$ 钉在零均值上; 更新延后一步生效, 一批数据不会用自己算出的 bias 路由自己



对比固定步长

DeepSeek-V3 式更新 $b_j \leftarrow b_j + \gamma \operatorname{sign}(\bar{\ell} - \ell_j)$ 只用到负载的符号: γ 小则跟不上分布漂移, γ 大则负载来回震荡; 专家池涨到 896 之后这个可用区间几乎消失。

QB 用到的是分数的完整分布, 一步定位, 与 n 无关地稳定。代价是每步要对每列做一次分位数 (可用 `torch.kthvalue` 之类, $O(mn)$)。

注意「重新路由后恰好 $(2, 2, 2, 2)$ 」不是定理: 定理只保证门槛固定时每列恰好 q 个。真正路由时门槛会随新偏置一起动, 所以实际负载是近似均衡——这里挑了一个正好对上的例子。

轴	m : 一个训练批里的 token 数 (图中 8, 实际是整批); n : 路由专家数 (图中 4, K3 是 896); k : 每 token 激活数 (图中 1, K3 是 16)。 $\mathbf{S}$ 、 $\mathbf{S} + \mathbf{1b}^\top$ 、 $\mathbf{M}$ 、 $\tilde{\mathbf{M}}$ 四个面都是 $m \times n$ 且严格等大; $\boldsymbol{\alpha}$ 是 $m \times 1$ 竖条; $\mathbf{b}$ 、 $\widehat{\mathbf{b}}$ 是 $1 \times n$ 横条, 列宽与矩阵一致, 广播关系可直接对齐读。
对象	$\mathbf{s}_i \in (0, 1)^n$: Sigmoid 路由分数, 各专家独立、不归一化。 $\mathbf{b} \in \mathbb{R}^n$: 每专家一个偏置, 代码里是 <code>gate.e_score_correction_bias</code> ; 色深只表示 $ b_j $, 正负未编码 , 具体数值写在下方。 $\boldsymbol{\alpha}^{(t)} \in \mathbb{R}^m$: 每 token 一条门槛, 取自有偏分数。 $\mathbf{M}$: margin, 有正有负, 色深按值线性映射。 橙圈 = 被路由到的格 (一个整数选择), 紫框 = 贡献门槛的第 $k+1$ 格。
机制	(i) 三处「取自哪个分数」必须分清: 路由与门槛取自有偏分数 $\mathbf{s} + \mathbf{b}$; margin 与混合权重 $p_{i,j}$ 取自原始分数 $\mathbf{s}$ 。所以旧偏置只通过 $\boldsymbol{\alpha}$ 进入更新, 也不会污染 p 的梯度。 (ii) 单调性是全部: 负载 $\sum_i \mathbf{1}[M_{ij} > -\widehat{\mathbf{b}}_j]$ 关于阈值单调递减, 把「求一个使负载 = q 的偏置」化成「求一个分位数」, 一次前向即可, 无需辅助损失、也无需迭代逼近。 (iii) 减均值这一步纯粹是规范化: 给所有专家分数同时加一个常数不改变逐行 Top- k , 但能防止 $\mathbf{b}$ 整体漂走。